0. İÇİNDEKİLER

0. İÇİNDEKİLER..... 1

1. GİRİŞ 4

2. KURAMSAL BİLGİLER..... 5
2.1 Visual Studio.Net..... 5
2.1.1 Visual Studio.Net Nedir?..... 5
2.1.2 Visual Studio.Net İle Çalışmak..... 5
2.1.2.1 Proje Açmak..... 6
2.1.2.2 Properties Window ve Dynamic Help..... 7
2.1.2.3 ToolBox..... 8
2.1.3 NameSpace Mimarisi..... 8
2.1.4 .Net FrameWork..... 9
2.2 SQL Server..... 9
2.2.1 SQL Server Nedir?..... 9
2.2.2 SQL Server Kullanarak Neler Yapılabilir?..... 10
2.2.3 Veri Saklama Modelleri.....10
2.2.3.1 OLTP Veritabanları.....10
2.2.3.2 OLAP Veritabanları.....10
2.2.4 Client Uygulamaları.....10
2.2.4.1 Transact-SQL.....10
2.2.4.2 XML.....10
2.2.4.3 MDX.....11
2.2.4.4 OLEDB ve ODBC API'leri.....11
2.2.4.5 ActiveX Data Objects ve ActiveX Data Objects 11
2.2.4.6 English Query.....11
2.2.5 Client – Server Bileşenleri.....11
2.2.5.1 Client – Server Mimarisi.....11
2.2.5.2 Client Bileşenleri.....12
2.2.5.2.1 Client Applications.....12
2.2.5.2.2 Database API.....12
2.2.5.2.3 Client Net-Library.....12
2.2.5.3 Server Bileşenleri.....12
2.2.5.3.1 Server Net-Library.....12
2.2.5.3.2 Open Data Services.....12
2.2.5.3.3 Relational Engine.....12
2.2.5.3.4 Storage Engine.....13
2.2.6 Client-Server Haberleşme İşlemleri.....13
2.2.7 SQL Server Servisleri.....13
2.2.7.1 MSSQLServer Service.....13
2.2.7.2 SQLServerAgent Service.....14
2.2.7.3 Microsoft Distributed Transaction Coordinator.....14

2.3 ADO.NET.....14
2.3.1 ADO.NET Nedir?.....14
2.3.2 ADO.NET Avantajları..... 16
2.3.2.1 ADO.NET ile ADO Karşılaştırması.....16
2.3.3 ADO.NET Gelişimi.....17
2.3.4 ADO.NET Mimarisi.....17
2.3.5 ADO.NET Nesneleri.....18
2.3.5.1 Connection.....18
2.3.5.1.1 SqlConnection.....19
2.3.5.1.2 OleDbConnection.....20
2.3.5.2 DataSet.....20
2.3.5.2.1 DataSet Nedir.....20
2.3.5.2.2 Nasıl Kullanılır?.....21
2.3.5.2.2.1 DataSet Oluşturma.....21
2.3.5.2.2.2 Kayıt Ekleme.....22
2.3.5.2.2.3 Değişiklikleri Kabul Etme.....22
2.3.5.2.2.4 Veri Bileşenlerini Bağlama.....22
2.3.5.3 DataAdapter.....23
2.3.5.3.1 SelectCommand Metodu.....23
2.3.5.3.2 UpdateCommand Metodu.....24
2.3.5.3.3 InsertCommand Metodu.....24
2.3.5.3.4 DeleteCommand Metodu.....24
2.3.5.4 Data Reader.....25
2.3.5.4.1 DataReader Nedir?.....25
2.3.5.4.2 DataReader Oluşturulması.....25
2.3.5.4.3 Read Metodu.....26
2.3.5.4.4 Çoklu Sorguların Çalıştırılması.....27
2.3.5.5 Command Nesnesi.....27
2.3.5.5.1 Command Nedir?.....27
2.3.5.5.2 Command Oluşturma ve Kullanma.....27

3. UYGULAMA.....28
3.1 Kullanıcı Arayüzü 28
3.1.1 Menu Formu.....28
3.1.1.1 Menü Formunun Görünümü ve Kullanımı.....28
3.1.1.2 Menü Formuna Ait Kod açıklaması.....29
3.1.2 Firma Kayıt Formu.....30
3.1.2.1 Firma Kayıt Formunun Görünümü ve Kullanımı.....30
3.1.2.2 Firma Kayıt Formuna Ait Kod Parçası 33
3.1.2.2.1 form_Load.....33
3.1.2.2.2 cboSemt_Click.....35
3.1.2.2.3 btnKaydet_Click.....35
3.1.3 Firma Listele Formu.....37
3.1.3.1 Firma Listele Formunun Görünümü ve Kullanımı.....37

3.1.3.2 Firma Kayıt Formuna Ait Kod Parçası……38
3.1.3.2.1 frmFirmaListele_Load…….38
3.1.3.2.2 btnListele_Click……...39
3.1.4 Eleman Kayıt Formu…….40
3.1.5 Sözleşme Kayıt Formu……..41
3.2 Veritabanı Tabloları……42
3.2.1 Sektör Tablosu(tblsektor)……..42
3.2.2 Alt Sektör Tablosu(tblaltsektor)……43
3.2.3 Semt Tablosu(tblemt)……43
3.2.4 Sokak Tablosu(tblsokak)……...44
3.2.5 Eleman Tablosu(tbleleman)……...45
3.2.6 Sözleşme Tablosu(tblsozlesme)……..…45
3.2.7 Müşteri Tablosu(tblmusteri)……..46
3.3 Tablolar Arası İlişkiler……48

4. SONUÇ VE ÖNERİLER…….…….49

5. KAYNAKLAR……..…….…..50

1 - GİRİŞ

Bilgisayar dünyasının hızla geliştiği ve değiştiği günümüz dünyasında, artık projeler farklı bir boyut kazanmıştır. Bununla birlikte bu projeleri geliştirecek platformların da sürekli gelişerek değişmesi, özellikle de bir Bilgisayar Mühendisi adaylarını sürekli yeni Platformlar ve yeni diller öğrenmeye ve bu gelişmleri takip etmeye itmiştir. Microsoft'un Visual Studio 6.0' ın devamı niteliğinde geliştirdiği Visual Studio .Net, kullanıcı dostu arayüzü, kod yazmada sağladığı kolaylıklar ve ADO.NET ile veritabanı işlemlerine getirdiği bağlantısız model yaklaşımıyla bir çok insan tarafından kullanılmaya başlamıştır. Bu projede Visual Studio .Net platformunda bir windows uygulaması geliştirilerek hem Visual Studio .Net hakkında yeteri kadar bilgi sahibi olmak, hem de Windows uygulamaları hakkında deneyim kazanabilmek amaçlanmıştır.

Proje sadece Tekstil sektörüne Web sayfası ve Portal tasarımı hizmeti veren bir bilgisayar firması için yapılmıştır. Bilgisayar firması reklam politikası olarak İstanbul'daki Tekstil merkezlerindeki (Osmanbey, Laleli, Zeytinburnu, Merter v.s.) firmalara broşür dağıtarak, her broşür karşılığında firmalardan bir kartvizit alınması gibi bir yol izlenmektedir. Proje, önceleri, toplanılan bu kartvizitlerin bir veritabanında tutulması ve gerektiği zaman arama yapılıp, gerekli istatistik bilgilere ulaşılabilmesi için yapılmıştır. Daha sonra bilgisayar firmasında çalışan elemanların ve firmanın yaptığı işlerin de istatistik bilgilerine ulaşabilmek ve bir arşiv oluşturabilmek amacıyla geliştirilmiştir. Bu işlemlerin yapılabilmesi için Visual Studio .Net platformu ve Visual Basic .Net dili kullanılarak kullanıcı arayüzü tasarlanmıştır. Bu arayüzde kayıt, arama ve listeleme formları geliştirilmiştir. Proje Windows 2000 Proffessional iletim sistemi üzerinde MS Access 2000 veritabanı kullanılarak yapılmıştır.

2 – KURAMSAL BİLGİLER

2.1 Visual Studio.Net

Bu kısmda Visual Studio.Net nedir ve nasıl kullanılır ve getirdiği yenilikler anlatılacaktır.

2.1.1 Visual Studio.Net Nedir?

Visual Studio .Net , .Net platformu için geliştirilmiş uygulama geliştirme platformudur. En büyük özelliklerinden birisi IDE (Integrated Development Environment) dediğimiz ortak bir uygulama geliştirme platformu sunmasıdır. İster C# ile uygulama geliştirin ister VB.NET ile, kullanacağınız en etkin ve ortak uygulama geliştirme platformu Visual Studio .Net olacaktır.

2.1.2 Visual Studio.Net ile Çalışmak

Visual Studio .Net platformunun kullanıcı dostu arayüzü, bu platformda proje geliştirmeyi daha kolay ve zevkli kılıyor. Visual Studio .Net arayüzü aşağıdaki şekilde gösterilmiştir.

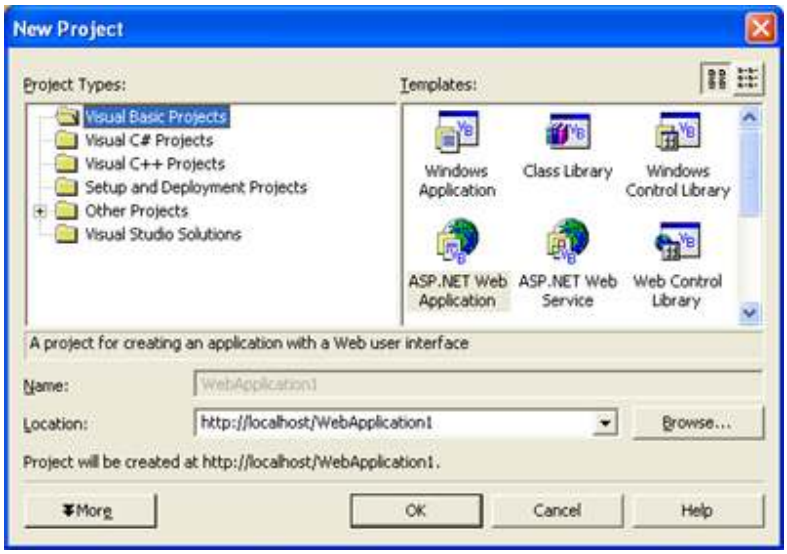
Şekil 2.1. Visual Studio .Net Açılış Ekranı

Visual Studio .Net'i açtığımızda karşımıza gelen ilk ekranda, Start başlığı altında yer alan ortadaki ana sayfa aslında dikkat ettiğinizde bir web sayfasıdır. Sol menüde **'What's New'**, **'Online Community'**, **'Downloads'** gibi başlıklar bulunmaktadır. Bu linklerin ardında Visual Studio .Net'in web sitesi var ve kullanıcıya tüm yenilikleri, internet üzerinde oluşan gruplara erişimi, hatta yazılımın son download seçeneklerini tek ekranda yani bu anasayfada sunuyor. **'My Profile'** linki altında ise kullanıcıya birkaç küçük soru sorarak, tüm ekran görüntülerini kullanıcıya özel şekillendiriyor ve orta kısımda da son açılan projeleri listeleyen bir kısım var.

2.1.2.1 Proje Açmak

Visual Studio .Net ilk açıldığında karşımıza çıkan pencerede New Project linkine tıklayarak yeni bir proje başlatabiliriz veya Open Project linkine tıklayarak daha önce oluşturduğumuz projeleri açabiliriz. Aynı zamanda son uğraştığımız projeler de ekranda göründüğü için doğrudan proje ismini seçerek istediğimiz projeyi açarız.

Yeni Proje oluşturmak için New Project linkine tıkladığımızda karşımıza çıkan ekran aşağıda gösterilmiştir.



Şekil 2.2. Visual Basic ile ASP.NET Web Uygulaması

Yukarıdaki pencerede görüldüğü gibi, sol kısımda hangi dilde proje geliştirileceği seçilir ve yapabileceğimiz projenin içeriğinde de 'Windows Application' yani işletim sistemi üzerinde çalışabilecek bir program(exe), 'Class Library' yani program modülleri için kütüphaneleri, ayrıca ASP.NET Web Uygulamaları ve Web Servisleri gibi seçenekler bulunur. Yukarıdaki ekran, proje tipi olarak Visual Basic Projects seçildiğinde karşımıza çıkar. Eğer proje tipi olarak C# seçilirse projenin içeriği kısmında çıkan seçeneklerin yine aynı olduğu görülür. Yani birden farklı dil kullanılarak aynı türde proje oluşturulabilir.

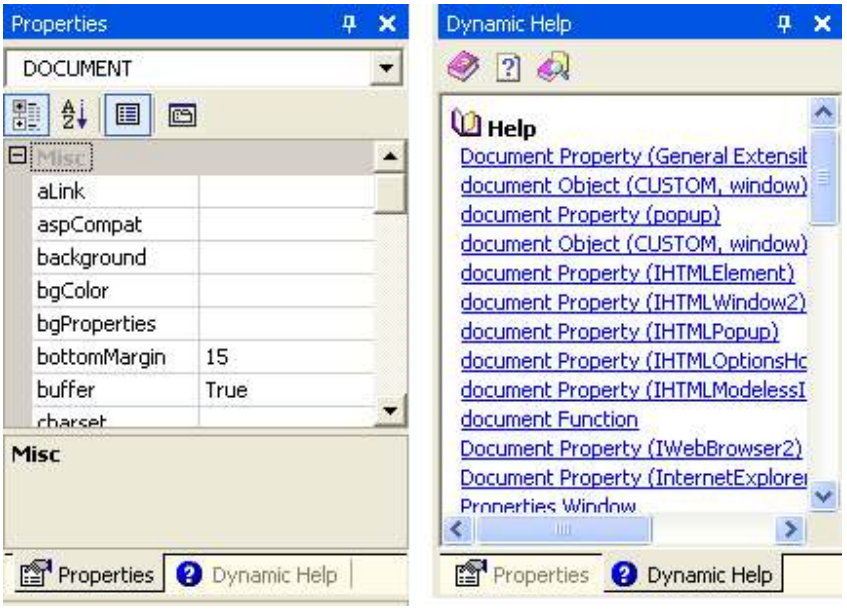
Şekil 2.3. Visual C# Projesi

Yukarıdaki pencerede görülen Name kısmı projenin adını ve Location kısmı da projenin nereye kaydedileceğini gösterir.
Eğer Proje Türü olarak ASP.Net Web uygulaması seçilirse, otomotik olarak proje için Internet Information Server altında bir Virtual Directory açılacak ve dosyalar belirlenmiş web klasörü altında yer alacak.

Şekil 2.4. ASP.Net Projesi Oluşturuluyor

2.1.2.2 Properties Window ve Dynamic Help

Visual Studio .Net'in daha önceki veriyonlarında da bulunan **Properties** yazılım üzerinde bulunulan yere göre değişen özellikler menüsüne sahiptir. Her hangi bir Control'ün özellikleri bu pencere kullanılarak ayarlanabilir. Ayrıca Visual Studio .Net'in ile birlikte gelen en önemli özelliklerden birisi de **Dynamic Help** özelliğidir. Dynamic Help sayesinde o an üzerinde çalışılan nesne ve özellikleri ile ilgili yardım alınabilir.Aşağıda bu iki özellikle ilgili şekiller verilmiştir.



Şekil 2.5. Properties ve Dynamic Help Pencereleri

2.1.2.2 ToolBox

Visual Basic ve Visual C++ gibi yazılım geliştirme ortamlarında da bulunan ToolBox Visual Studio .Net içerisinde de bulunmaktadır. ToolBox kullanarak oluşturmak istediğimiz Formları istediğimiz gibi şekillendirebiliriz.

2.1.3 Visual Studio .Net FrameWork[6]

2.1.4 Visual Studio .Net NameSpace Mimarisi[6]

2.2 SQL Server

2.2.1 SQL Server Nedir?

SQL Server'ı iki tür veritabanını yönetmek için kullanırız. Bunlar OLTP (Online Transaction Processing) veritabanları ve OLAP (Online Analytical Processing) veritabanları. Genel olarak farklı client'lar network üzerinden haberleşerek veritabanlarına erişirler. SQL Server ile terabyte boyutundaki veritabanlarını yönetebilirsiniz. Birden fazla server arasında Windows Clustering yaparak SQL Server'ı kullanabilirsiniz.

2.2.2 SQL Server Kullanarak Neler Yapılabilir?

SQL Server ile yoğun verileri işleyebilir, saklayıp analiz edebilir ve yeni uygulamalar geliştirebilirsiniz. SQL Server OLTP ve OLAP için gerekli olan veri saklama ürünlerini ve teknolojilerini destekler. SQL Server bir ilişkisel veritabanı yönetim sistemidir (RDBMS: Relational database management system).SQL Server;

- Veri işleme ve analiz için saklanan veri yığınlarını yönetebilir.
- Client uygulamalarından gelen isteklere cevap verebilir.
- SQL Server ve clientlar arasında veri göndermek için T-Sql (Transact SQL), XML, MDX veya SQL-DMO kullanabilir.[1]

2.2.3 Veri Saklama Modelleri

2.2.3.1 OLTP Veritabanları

Bir OLTP veritabanı içinde veriler genellikle ilişkisel tablolar içinde organize edilir. Gereksiz veri yığınları azaltır ve veri güncelleme hızını arttırır. SQL Server çok sayıda kullanıcının gerçek zamanlı olarak veri analiz edebilmesini ve güncellemesini sağlar. Örnek olarak OLTP veritabanları havayolu bilet satış bilgileri ve bankacılık işlemlerini içerir.

2.2.3.2 OLAP Veritabanları

OLAP teknolojisi büyük verilerin organize edilmesi ve incelenmesini sağlar. Örneğin bir analist büyük verileri hızlı ve gerçek zamanlı olarak değerlendirebilir.SQL Server Analiz Servisi toplu raporlama ve analizde, veri modelleme ve karar desteğe kadar geniş alanda çözümler sunar.[1]

2.2.4 Client Uygulamaları

2.2.4.1 Transact – SQL

Bu sorgulama dili SQL'in farklı bir versiyonudur. SQL Server kullanıcıları için birincil bir sorgulama ve programlama dilidir.

2.2.4.2 XML

Bu format bir sorgu ve prosedürün çalışması sonucu gelen verinin HTTP üzerinden URL veya şablonlar kullanılarak iletilmesidir. XML veritabanina veri girerken, güncellerken ve silerken kullanılabilir

2.2.4.3 MDX

MDX bir OLAP veritabanı içindeki çok boyutlu objelerin, sorguların ve çok boyutlu verilerin düzenlenmesini ifade eder.

2.2.4.4 OLEDB ve ODBC API'leri

Client uygulamaları veri tabanına komut gönderirken OLE DB ve ODBC API'lerini kullanırlar. Bu API'leri kullanarak gönderilen komutlar T-SQL dilini kullanırlar.

2.2.4.5 ActiveX Data Objects ve ActiveX Data Objects (Multidimensional)

ADO ve ADO MD Visual Basic, ASP, VB Script programlama dillerinde OLE DB'yi kapsamak için kullanılır. OLTP veritabanları içindeki verilere erişmek için ADO kullanılır. Analiz Servisi veri küpleri içindeki verilere erişmek için ise ADO MD kullanılır.

2.2.4.6 English Query

Bu uygulama ile kullanıcılar doğal dillerini kullanarak karmaşık T-SQL ve MDX yapılarını kurarak veritabanıdan bilgi elde edebilirler. Örneğin bir kullanıcı "Satış bölümünün bu ay ki cirosu nedir ?" diye bir soru sorabilir. Adındanda anlaşıldığı gibi sorularda ingilizce dili kullanılır.

2.2.5 Client – Server Bileşenleri

SQL server verileri saklamak ve tekrar almak için client, server bileşenlerini içerir. SQL server uygulamaları temel network ve protokollerden izole etmek için katmanlı iletişim modelini kullanır.Bu yapı uygulamanız farklı network ortamlarında kullanmanızı sağlar.

2.2.5.1 Client – Server Mimarisi

SQL server iş yükünü azaltmak için server ve clientlar üzerindeki iş yükünü parçalara böler.

· Client uygulama ticari hesaplama ve bunların kullanıcıya gösterilmesinden sorumludur. Bunlar her ne kadar client üzerinde çalışsa bile veri sorgularken server üzerinde çalışır.

· SQL Server birden fazla gelen istekleri karşılayabilmek için disk, bellek, işlemci ve bant genişliği gibi kaynakları yönetebilir.

Client-Server mimarisi ile çeşitli ortamlar için uygulamalar dizayn edip geliştirebilirsiniz. Client uygulaması çeşitli bilgisayar ortamlarında çalışır ve SQL Server ile network üzerinden haberleşir.

2.2.5.2 Client Bileşenleri

2.2.5.2.1 Client Applications

Bir client uygulamasındaki amaç T-SQL kodlarını göndermek ve sonuçları almaktır. Veritabanı API'si kulanarak bir client uygulaması geliştirebilirsiniz. Böylece SQL Server ile haberleşmek için kullanılan network protokolünü bilmeye gerek yoktur.

2.2.5.2.1 Database API

Database API'leri bir driver veya DLL kullanarak T-SQL kodlarını geçirir ve sonuçları alır. Bu T-SQL kodlarını gönderirken ve sonuçları alırken client uygulama ile SQL Server arasında kullanılan bir arayüzdür.

2.2.5.2.1 Client Net-Library

Client Net-Library network bağlantılarını control eder ve verileri client'a yönlendirir. Bu bir çeşit veritabanı isteklerini ve sonuçlarını paketleyip çeşitli network ortamlarında taşıyan haberleşme yazılımıdır.[1]

2.2.5.3 Server Bileşenleri

2.2.5.3.1 Server Net-Library

SQL Server aynı anda birçok Net-Library'leri izleyebilir. Bağlantının sağlanabilmesi için Client Net-Library ve Server Net-Libraries aynı olmalıdır. SQL Server şu protokollere destek verir; TCP/IP, Named Pipes, NWLink, IPX/SPX, VIA ServerNet II SAN, VIA GigaNet San, Banyan VINES ve AppleTalk.

2.2.5.3.2 Open Data Services

SQL Server'ın bir bileşeni olan bu uygulama network bağlantılarını izler, client isteklerini çalıştırmak üzere SQL Server'a geçirir ve sonuçları tekrar client uygulamaya döndürür. Open Data Services SQL Server'a yüklenmiş olan tüm Net-Libraries'i dinler.

2.2.5.3.3 Relational Engine

T-SQL kodlarını çalışabilir parçalara böler, optimizasyonunu yapar, tanımlanmış diğer kodları çalıştırır ve güvenliği sağlar.

2.2.5.3.4 Storage Engine

Storage Engine veritabanı dosyalarını, dosyalardaki boş alanları, fiziksel sayfalara veri giriş çıkışını, veri önbelleklerini, loglamaları, kurtarma operasyonlarını yönetir.

2.2.6 Client-Server Haberleşme İşlemleri

Client ve Serverlar tipik olarak bir network üzerinden haberleşirler. Aşağıdaki adımlar bir sorgunun veritabanı API'si kullanılarak hangi aşamalardan geçtiğini göstermektedir.

1. Bir client uygulaması sorgu gönderir. Client uygulama API'yi çağırır ve sorguyu verir. API driver veya DLL'I kullanarak sorguyu bir veya daha fazla TDS (Tabular Data Stream) paketlere gruplar ve Client Net-Library'e geçirir.

2. Client Net-Library bu paketleri networkte taşınabilecek şekilde paketler. Client Net-Library windows interprocess communication (IPC) API'sini çağırır ve işletim sisteminde yüklü olan protocol üzerinde bu paketleri server net-libraries'e gönderir. Server net-libraries bu network paketlerini çözer ve TDS paketlerinin elde eder. Data sonar bu TDS paketlerini Open Data Services'e geçirir.

3. Open Data Services bu TDS paketlerini açar ve sorguyu relational engine'e verir. Relational engine bu sorguyu derler ve optimizasyonu yapılmış çalışabilir bir şekler çevirir. Ve çalıştırır. Relational engine storage engine ı ile OLE DB arayüzünü kullanarak haberleşir.

4. Storage engine databaseden dönen verileri data buffer'a oradan relational engine'e verir. Relatinal Engine gelen verilere son şeklini verir ve Open Data Services'e geçirir.

5. Open Data Services bu verileri Server Net-libraries, network protokolü, client net-library ve database API kullanarak client'a iletir. Bu sonuçlar XML şeklinde de iletilebilir.[1]

2.2.7 SQL Server Servisleri

Microsoft SQL Server aşağıdaki servisleri içerir;

MSSQLServer service, SQLServerAgent service, Microsoft Distributed Transaction Coordinator (MS DTC) ve Microsoft Search. Bu servisler genellikle Windows 2000 üzerinde servis olarak çalışırlar fakat uygulama gibi de çalışabilirler.

2.2.7.1 MSSQLServer Service

MSSQLServer bir veritabanı motorudur. Tüm T-SQL yapılarını çalıştıran ve veritabanını kapsayan tüm dosyaları yöneten servistir. MSSQLServer servisi;

1. Sistem kaynakları birden fazla kullanıcıya paylaştırır

2. Mantıksal hataları engeller. Mesela bir datayı aynı anda güncellemek isteyen kişileri engeller.

3. Veri bütünlüğünü sağlar.

2.2.7.2 SQLServerAgent Service

Bu servis SQL Server ile birleşik olarak çalışır ve alert'leri ve multiserver işlemlerin yönetilmesini sağlar.

1. Alert'ler bir işlemin sonuçları hakkında bilgi verir. Mesela bir sorgu bitti veya çalışma sırasında bazı hatalarla karşılaşıldı gibi.

2. SQLServerAgent görev oluşturma ve zamanlama aracı ile bazı işlemlerin otomatikleştirilmesini sağlar.

3. SQLServerAgent servisi bir problem olduğunda mail atabilir, çağrı cihazına mesaj gönderebilir veya başka bir uygulamayı çalıştırabilir. Mesela bir veritabanı dolduğunda veya bir yedekleme işlemi bittiğinde size mail atmasını sağlayabilirsiniz.

2.2.7.3 Microsoft Distributed Transaction Coordinator

MS DTC bir işlem ile birden fazla farklı kaynağın üzerinde işlem yapılmasını sağlar. Mesela bir işlem ile tüm serverlar üzerinde kalıcı bir güncelleme işlemi yapabilir veya yapılmış bu işlemi hepsinden geri alabilir.

2.3 ADO.NET

2.3.1 ADO.NET Nedir

ADO.NET; Microsoft' un RDO, DAO ve ADO' dan sonra bizlere sunduğu .NET ailesinden gelen yeni bir veriye erişim modelidir. ADO.NET bağlantısız veri kümeleri ile çalışmak için tasarlanmıştır. Bağlantısız veri kümeleri ağ trafiğini azaltmaktadır.[2]

ADO.NET evrensel iletim biçemi olarak XML'i desteklemektedir. Böylece, iletilen bir bileşenin XML ayrıştırıcısına sahip olması koşuluyla herhangi bir platform üzerinde

çalışması sağlanarak platformdan bağımsız çalışabilirlik garanti altına alınmış olur. İletim XML üzerinden gerçekleştirildiğinde, alıcının artık bir COM nesnesi olması gerekliliği ortadan kalkmaktadır. Alıcı bileşenle ilgili mimariye dayalı ya da başka herhangi bir kısıtlama yoktur. Herhangi bir yazılım bileşeni ADO.NET verisini iletim biçemi olarak aynı XML schema'sını kullandığı sürece paylaşabilir.

ADO.NET kullanılarak güvenilir, ölçeklenebilir ve XML kullanabilen uygulamalar geliştirilebilir. ADO.NET ve XML sınıfları tek bir mimarinin bileşenleri olarak birbiriyle sıkı ilişki içinde geliştirilmişlerdir.

ADO.NET, OLE DB ve XML üzerinden sunulan veri kaynaklarına olduğu gibi, Microsoft SQL Server gibi veri kaynaklarına da tutarlı veri erişimi sağlamaktadır.

Şekil 2.6. ADO.Net Mimarisi[2]

2.3.2 ADO.NET Avantajları

Hatırlayalım ADO ile veri alış verişi aktif bir bağlantı üzerinden gerçekleşirdi. Her yeni oturum aktif yeni bir bağlantı demekti. Kullanıcı potansiyeli düşük bir yapının omuzlarına fazla bir yük binmeyeceği için, oluşabilecek aksaklıkları hissedilmeyebilir. Fakat Kurumsal yapılarda durum farklı. Her gün binlerce ziyaretçisi olacak olan bu yapıda ortaya çıkabilecek bir çok sorun olacaktır. Bunların başında yaşanan yavaşlıklar kullanıcıların ve yöneticinin epey başını ağrıtacaktır. ADO.NET'in bu tür sorunlara getirdiği çözüm mükemmel. Veri alışverişinde bağlantısız bir erişim sağlıyor. Ayrıca veri işlemlerini yaparken veri kaynağına en az seviyede erişerek işlemleri yapmamızı sağlayan DataSet nesnesi de ADO.NET ile gelen yeniliklerden biri.

ADO'da veritabanı bağlantılarını OLEDB sağlayıcıları aracılığıyla gerçekleşiyordu. Veri tabanı bağlantılarımızı yaparken SQL Server' ı da kullansak Access' ı da kullansak aynı hattı kullanmak gerekiyordu. ADO.NET' le birlikte ilk göze çarpan SQL Server' la doğrudan konuşabilen nesne grubunun gelmesi. Bunun yanında klasik OLEDB bağlantısını da rahatlıkla yapabiliyoruz .

ADO ile ADO.NET arasındaki XML mimarisinde de önemli farklılıklar vardır. ADO da kullanılan yapı COM Marshalling mimarisine dayanıyordu. COM Marshalling mimarisi bir bileşenden başka bir bileşene Recordset geçirilirken kullanılıyordu. Ayrıca bu modelde verinin anlam kazanması için farklı yöntemler kullanmak gerekiyordu. Bu modelin bir özelliği de COM bileşenlerinde kullanılabilmesiydi. Bu nedenle sadece COM platformundaki veri türlerini destekliyor. ADO.NET' teki yapısı ise tamamen XML'e dayanmaktadır. DataSet ile veri kaynağı arasındaki ilişki tamamen XML teknolojisine

dayanmaktadır.ADO'da desteklenmeyen türler bu yapıda desteklenmektedir.[3]

.NET ailesi veriye erişim için iki sağlayıcı içermektedir. Bunlardan ilki OLEDB.NET sağlayıcısı, ikincisi ise SQL Server.NET. OLEDB.NET' in eskiden kullandığımız OLEDB'nin gelişmiş versiyonu olduğunu söyleyebiliriz. SQL Server.NET' in en önemli özelliği SQL Serverla direk olarak konuşmasıdır. Bu özellik bize büyük performans artışı sağlamaktadır.

Bir başka özellikte bu sağlayıcıların framework sınıf kütüphanesinde farklı Namespace' lerle temsil ediliyor olması. OLEDB.NET sağlayıcısı için System.Data.OleDb, SQL Server.NET içinse System.Data.SqlClient kullanılmaktadır.

2.3.2.1 ADO.NET ile ADO Karşılaştırması[2]

Tablo 2.1. ADO ile ADO.NET Karşılaştırılması

ÖZELLİK	ADO	ADO.NET
Hafızada kalan veri	**Tek tablo gibi davranana Recordset nesnesini kullanır.**	**Birden fazla tablo içerebilen DataSet nesnesini kullanır.**
Çoklu tablolar arası ilişki	**Farklı veritabanındaki tabloları tek bir sonuç tablosunda göstermek için Join sorgusunu kullanır.**	**Herhangi bir tablodaki kayıtları başka tablolardaki kayıtlarla ilişki kurabilmek üzere tasarlanmış DataRelation nesnesini destekler.**
Veriye erişim	**Ardışık olarak RecordSet satırlarını tarar.**	**Ardışık olmayan erişimlere olanak sağlar.Satırlar arası ilişkileri takip ederek farklı satırları tarayabilir.**
Çevrimdışı erişim	**RecordSet nesnesi tarafından desteklenmesine rağmen verim alabilmek için genellikle sürekli bağlantı kullanılır.**	**DataSet nesnesi standart olarak bu yapıyı destekler.**
Programlanabilirlik	**Komutları Connection nesnesini kullanarak iletir.**	**Xml'in titizlikle hazırlanmış programlama özelliklerini kullanır. Xml'de bilgi kendini tanımlar. Yapının temelini oluşturan temeller (tablo ve kayıtlar gibi) Xml'de kolay okumayı sağlamak için, görünmez.**
Çevrimdışı verinin bileşenler veya diziler arasında paylaşımı	**COM dizilemesini kullanılır.Bu yüzden sadece COM standartlarının tanımladığı veriler kullanılabilir.**	**DataSet'i Xml dosyası kullanarak taşır.Xml formatı veride herhangi bir kısıtlama koymaz ve herhangi bir tip dönüşümü gerektirmez.**
Verinin firewall'lardan (ateş duvarları)	**Doğuştan problemlidir.Çünkü firewall'lar zaten sisteme**	**Herhangi bir sorun çıkarmaz. Çünkü DataSet nesnesi yazı bazlı**

geçirilmesi	**ulaşma isteklerine (COM nesneleri gibi) karşı tasarlanmıştır.**	**Xml kullanır. Bu Xml dosyaları firewall'lardan rahatlıkla geçebilirler.**

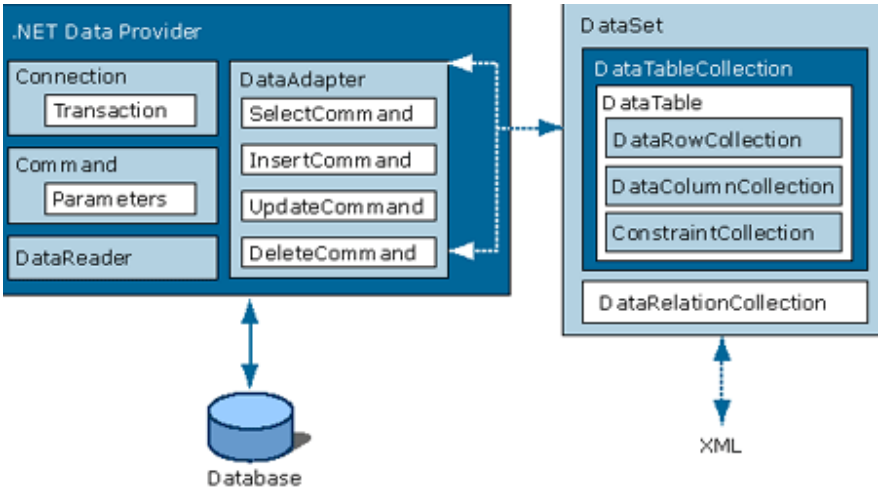
2.3.3 ADO.NET Gelişimi

Bilgisayar dünyasında bilgilerin kalıcı olarak saklanması her zaman en önemli meselelerden birini teşkil etmektedir.Bu bilgilerin saklanması için **database(veritabanları)** ler geliştirilmiş ve İnternetin hayatımızın her alanını kapladığı şu zamanlarda yaygın bir biçimde kullanılmaktadır. Tabii ki veritabanlarının yoğun bir şekilde kullanılmaya başlamasıyla birlikte, veritabanına ulaşmak için kullanılan yolların performans durumu gündeme geldi ve ADO.Net buna bir çözüm olmak amacıyla bağlantısız model mantığı kullanılarak geliştirilmiştir.

2.3.4 ADO.NET Mimarisi

ADO.NET mimarisini ikiye ayırarak incelemek gerekiyor. Biri **bağlantılı**, diğeri **bağlantısız** taraf. Bağlantılı taraf, bir .NET veri sağlayıcısının üzerinden yapılan bağlantıyla çalışan nesneleri içeriyor. **Connection**, **Command**, **DataRader**, **DataAdapter** bu nesnelerin başlıcaları. Bu nesnelerin ortak özelliği, aktif bir bağlantıya ihtiyaç duymaları. Diğer tarafta, bağlantısız bir yapı var: **DataSet**. DataSet, veri kaynağı bağlantısından bağımsız olarak tasarlanan bir nesne. Hafızada duran ve istendiğinde XML olarak kalıcı hale getirilebilen ilişkisel bir veritabanı olarak düşünebilirsiniz. Zira DataSet, birden fazla tablolar içerebiliyor; bu tablolar arası ilişkiler, kısıtlamalar, görünümler tanımlamanıza olanak veriyor.[4]

ADO.NET mimarisini en güzel şekilde aşağıdaki şekil ifade ediyor:



Şekil 2.7. ADO.NET Mimarisi

2.3.5 ADO.NET Nesneleri

2.3.5.1 Connection

ADO.NET'in veri kaynağıyla canlı ilişkide olan tarafını .NET Veri Sağlayıcısı oluşturuyor. Bu sağlayıcı, veri kaynağına gönderilecek her türlü komut, ve ondan dönecek her türlü yanıt için bir köprü vazifesi görüyor. Tüm bu operasyonlar için, bu sağlayıcının aktif bir bağlantısı olması gerekiyor. İşte bu bağlantı, **Connection** nesnesiyle temsil ediliyor. Connection nesnesi, ADO.NET'in en temel nesnesidir.

Her veri sağlayıcısı, kendine özgü gerçeklenmiş bir bağlantı nesnesi sunmak durumunda. .NET ile gelen OLE DB .NET sağlayıcısının sunduğu bağlantı nesnesi OleDbConnection. SQL Server .NET veri sağlayıcısının bağlantı nesnesi ise SqlConnection. Her iki nesnenin de kullanımı genel olarak aynı.[5]

2.3.5.1.1 SqlConnection

Microsoft SQL Server 7.0 ve üst sürüm veritabanlarına erişmek için **System.Data.SqlClient** aduzayı altındaki **SqlConnection** nesnesini kullanılır. Bu bağlantı nesnesi (daha doğrusu nesneyi sunan sağlayıcı), sadece SQL Server için tasarlandığı için başka bir veritabanı erişiminde kullanılmaz. SQL Server'in daha alt sürümlerine erişim içinse eskiden olduğu gibi yine OLE DB kullanılacak. Böyle bir zorunluluk olmamasına rağmen 7 ve üst versiyonlarında ayrı bir sağlayıcı kullanılır. Aslında OLE DB hala bir seçenek, ancak veritabanı erişimi gibi masraflı bir işte daha yüksek performans arzuluyorsak, tercihimiz SQL Server .NET veri sağlayıcısı olmalıdır. Çünkü bu sağlayıcı, SQL Server ile **TDS (Tabular Data Stream)** paketleri seviyesinde, yani SQL Server'in **anladığı dilde** haberleşiyor. Böylece OLE DB gibi bir tercümanı direk saf dışı bırakmış oluyorsunuz. Aşağıdaki şekil bu durumu özetliyor:

Şekil 2.8 SQL Server ve OLEDB Bağlantısı[3]

SQL Server Bağlantı Örneği

```
Dim sBaglantiCumlesi as String = "Server=(local); User id=sa; Password=;
Database=Northwind"

    // SqlConnection tipindeki bağlantı nesnemizi,
    //    SqlConnection sınıfının, bağlantı cümlesini parametre alan
```

```
    //    yapılandırıcısı (constructor) ile oluşturuyoruz.
Dim oConn as SqlConnection = New SqlConnection(sBaglantiCumlesi)

    //  bağlantımızın kurulması
    oConn.Open()
    Console.WriteLine("Northwind ile baglanti kuruldu")

    // .... burda Northwind üzerinde
    //      istediğiniz operasyonları yaptığınızı varsayalım.

    // işimiz bitince bağlantıyı "kesinlikle" kapatmalıyız.
    oConn.Close();
    Console.WriteLine("Northwind ile baglanti kesildi")
```

2.3.5.1.2 OleDbConnection

OleDbConnection nesnesi, System.Data.OleDb aduzayına bağlı, **OLE DB .NET veri sağlayıcısı** üzerinden bir veri kaynağına bağlantı tesis etmemizi sağlar. Kullanımı aynen yukarıda bahsettiğimiz SqlConnection nesnesi gibidir. Yalnız bu tip bağlantıda, bağlantı cümlesini oluştururken, hedef veri kaynağına uygun bir **Provider** seçmemiz gerekecektir. OLE DB veri sağlayıcısını kullanarak, sistemde OLE DB sağlayıcısı bulunan her veri kaynağına (*SQL Server, Oracle, Access, Indexing Service*) bağlanılabilir.

<u>SQL Server Bağlantı Örneği</u>

```
Dim sBagCumle as String = @"Provider=Microsoft.Jet.OLEDB.4.0; Data
Source=C:\data.mdb "
Dim oConn as OleDbConnection = new OleDbConnection(sBagCumle)

oConn.Open()
Console.WriteLine("Veritabanı ile baglanti kuruldu")
oConn.Close()
```

2.3.5.2 DataSet

2.3.5.2.1 DataSet Nedir?

DataSet nesnesi herhangi bir veri kaynağı ile ilişkilendirilmiş veri kümesini temsil eder. Bu veri kaynağı bir veritabanı olacağı gibi XML formatlı bir kaynakta olabilir. Nitekim DataSet nesnesinde tutulan veriler hafızada XML formatında tutulmaktadır. DataSet içindeki veriler aynen veri tabanında olduğu gibi tablolar şeklinde saklanır. Yani bir DataSet nesnesi bir yada birden fazla tablo barındırabilir. DataSet'in yapısındaki tablolar **DataTable** isimli sınıfla temsil edilir. Bu da DataSet'in DataTable türünden nesneleri olan bir kolekesiyona sahip

olduğunu gösterir. Bu koleksiyonun adı **Tables**' dır. Tables koleksiyonun Add() ve Remove() metotları kullanılarak DataSet'e yeni tablo eklenip çıkarılabilir. Tables koleksiyonundaki tablolara indeksleyici yardımıyla erişilebilir. İndeksleyincinin parametresi tablonun ismi olabileceği gibi tablonun DataSet'teki sıra numarası da olabilir.[6]

Her bir DataTable nesnesinin **Rows** ve **Columns** koleksiyonu bulunmaktadır. Rows koleksiyonu DataTable'daki kayıtların kümesidir. Bu kümedeki her bir kayıt ise **DataRow** isimli sınıfla temsil edilir. Bir kayıttaki veriler sütunlara ayrılmıştır. Bir kayıt birden fazla sütundan oluşmuştur. Dolayısıyla her bir kaydın ilgili sütununa erişmek için DataRow sınıfındaki indeksleyici kullanılır. DataTable'da olduğu gibi indeksleyicinin parametresi sütunun adı yada sıra numarası olabilir.

DataSet nesnesinde veri ile direkt ilişkili olan bu nesnelerin yanı sıra ilişkisel veri tabanı modelinin gerektirdiği yapıları barındırmak içinde çeşitli sınıflar vardır. Bu sınıflardan en önemlisi **DataRelation** ve **Constraint** sınıfıdır. İki tür Constraint(kısıt) sınıfı vardır. Bunlar **UniqueConstraint** ve **ForeignKeyConstraint** sınıflarıdır. Constraint sınıfları temel olarak verinin tablolara yerleştirilme kurallarını belirler. Örneğin UniqueConstraint tablodaki bir kaydın tekil olmasını sağlarken, ForeignKeyConstraint sınıfı birbirleriyle ilişkili tablolardan birinde meydana gelen değişikliğin diğerinde nasıl bir etki yaratacağını belirler. DataTable nesnelerinin Constraints isimli koleksiyonuna oluşturulan bu Constraint nesneleri aktarılarak kısıtların çalıştırılması sağlanır. Dikkat edilmesi gereken nokta kısıtların çalışması için DataSet'in **EnforceConstraints** özelliğinin true olması gerektiğidir. Diğer bir önemli sınıf ise DataRelation sınıfıdır. Bu yazıda DataRelation detaylı bir şekilde anlatılacaktır. Bundan sonraki yazımda ise Constraint sınıfları üzerinde duracağım.[6]

Yukarıda bahsi geçen bütün sınıflar verinin hafızada modellenmesi için gerekli olan sınıflardır. Tüm bunlar için verinin DataSet içine doldurulması gereklidir. DataAdapter bir DataSet'in yada DataTable' ın içini doldurmak için veri kaynağı ile bağlantı kurar. Bu işlemi yapan DataAdapter sınıfının Fill() metodudur. Bu metot parametre olarak bir DataSet nesnesi alabileceği gibi DataTable nesnesi de olabilir. Eğer paramatre bir DataSet nesnesi ise DataSet nesnesi içinde yeni bir DataTable nesnesi oluşturulur ve veriler bu nesneye aktarılır. Veri kaynağına DataAdapter ile erişirken dikkat etmemiz gereken önemli bir nokta var : Her DataAdapter nesnesi ile ancak bir DataTable nesnesi oluşturulabilir. Dolayısıyla birden fazla tablo içeren DataSet nesneleri ile çalışmak için birden fazla DataAdapter nesnesi oluşturmalıyız. DataSet içinde yapılan değşikliklerin orjinal veri kaynağında güncellenmesi içinde DataAdapter kullanılır. DataAdapter'in güncellenen verileri düzgün bir biçimde ele alabilmesi için UpdateCommand, InsertCommand ve DeleteCommand gibi özelliklerinin belirtilmesi gerekir. **Command** nesneleri bir SQL dizgesi ve **Connection** nesnesi ile kurulur.

2.3.5.2.2 Nasıl Kullanılır?

2.3.5.2.2.1 DataSet Oluşturma

DataSet nesnesinin bir örneği yaratılacağı zaman DataSet constructor'ı kullanılır. Yeni bir DataSet nesnesi yaratırken isterseniz parametresiz çağırarak taban constructor'ını kullanabilirsiniz veya kendiniz bu metodu overload edebilirsiniz bu durumda ise constructor'e DataSet nesnesinin adını parametre olarak gönderirsiniz. Aşağıda bu iki durumun kullanımı gösterilmiştir.

Dim dataSet as DataSet = new DataSet(); Taban constructor'ı kullanılmış

Dim dataSet as DataSet = new DataSet("ornekDataSet"); overload edilmiş

DataSet nesnesinin içini doldurnayı .Net data providerları bir arabirim işlevi görerek yapar. **DataSet** nesnesi veriyi tutar, veri üzerinde update(güncelleme), delete(silme), add(ekleme) işlevlerinin yapılmasına da izin verir. Ancak bu **DataSet** nesnesinin o sırada veritabanı ile bağlantısı olmadığı için Update metodu çağrıldığı zaman veri üzerinde yapılan değişiklikler **DataAdapter** nesnesi aracılığı ile veritabanına gönderilirler.

2.3.5.2.2.2 Kayıt Ekleme

Yeni bir DataRow nesnesi yaratarak DataSet'e yeni kayıt eklemesi yapılır. Bu ise DataTable nesnesinin NewRow metodu kullanılarak gerçekleştirilir. Bu metod ile boş bir row(sıra) yaratılır. Kayıt ekleme için basit bir örnek kod aşağıda belirtilmiştir.

```
Dim row as DataRow = dataSet.Tables[0].NewRow()

row["soyad"] = "soyad"

row["ad"] = "ad"

dataSet.Tables[0].Rows.Add(row)
```

2.3.5.2.2.3 Değişiklikleri Kabul Etme

DataSet üzerinde herhangi bir veri üzerinde değişklik yapıldığı zaman veride değişiklik yapıldığına dair işaretlenir. Eğer veri üzerinde yapılan değişkliğin doğruluğu onaylanırsa veri üzerinde yapılan değişiklikler kabul edilmek üzere DataSet nesnesinin AcceptChanges() metodu çağrılır. Ayrıca tüm DataSet'ler üzerinde yapılan değişiklikleri kabul etmek istemiyorsanız

DataRow yada DataTable'ın AcceptChanges() metodunu kullanabilirsiniz. Bu sayede veriler üzerinde tam bir yönetim söz konusu olabilir.[4]

Yalnız DataSet üzerinde yaptığınız değişiklikler veritabanını değiştirmez.(Eğer DataSet'inizin veritabanı bağlantısı kesilmişse ki performans açısından bağlantının kesilmiş olması daha yararlıdır.). Bu yüzden **DataAdapter** nesnesinin Update() metodunu kullanarak veritabanına sadece değişiklik yapılan kısımların gönderilmesi sağlanır. (Dikkat tüm veriler gitmiyor sadece değişiklikler gidiyor.)

2.3.5.2.2.4 Veri Bileşenlerini Bağlama

Yeni .Net bileşenleri(component) sayesinde verileri bağlamak hem Windows Formlarında hem de Web Formlarında oldukça basitleşti. DataSetleri Windows veya Web Formlarına bağlayarak DataSetin içindeki bütün verileri görmek mümkün.

Bu bileşenlerden bir tanesi olan DataGrid'ler çoklu DataTable nesnesini içlerinde barındırırlar. Bu DataTable'lerin her biri de çoklu data row nesneleri içerirler. DataGrid'in DataSource özelliğine sadece DataSet nesnesini bağlamamız DataSet'lere erişmemiz için yeterli olur. Tabii bir de DataSet içerisinden hangi DataTable nesnesinin gösterileceğini belirtmemiz gerekir. Aksi halde DataGrid üzerinde DataSet'in sahip olduğu bütün DataRow'lar gösterilir. Bu işlemi ise DataGrid'in DataMember özelliğini kullanarak yapabiliriz.

2.3.5.3 DataAdapter

DataAdapter nesnesi DataSet nesnesi ile veritabanı arasında tercümanlık yapar diyebiliriz. Veritabanından aldığı bilgileri DataSet içerisine doldurur ve DataSet den aldığı bilgileri veritabanına geri gönderir. Dört önemli metoda sahiptir:

SelectCommand

UpdateCommand

InsertCommand

DeleteCommand

SQLDataAdapter ve OLEDBDataAdapter olarak iki nesne kullanılabilir. MSSQL Server a bağlantı için SQLDataAdapter nesnesi, diğer veritabanlarına bağlantı için OLEDBDataAdapter nesnesi kullanılır.

2.3.5.3.1 SelectCommand Metodu

Bu metod kullanılarak veritabanından bilgiler 2 şekilde listelenir: ya DataAdapter'in bu metoduna doğrudan bir **Command** atanır, yada SelectCommand'in **Text** özelliğine doğrudan bir Sql ifdesi içeren string atanabilir.

Bir örnek ile açıklamaya çalışalım. Bir select ifadesi ile veritabanından bilgileri listeleyip DataSet içine dolduralım. Ardından bu bilgiyi DataGrid içerisinde gösterelim;

Bu işlem için Imports System.Data.SqlClient kütüphanesini kullanmamız gerekli. Örneğimizde MSSQL Server a bağlanıp, Northwind veritabanında Customers table'ındaki bilgileri göstereceğiz.

```
Dim baglanti As SqlConnection = New
SqlConnection("server=localhost;uid=sa;pwd=;database=northwind")
Dim cmd As SqlCommand = New SqlCommand("SELECT CustomerID,
CompanyName FROM Customers", baglanti)
Dim adapter As SqlDataAdapter = New SqlDataAdapter()
adapter.SelectCommand = cmd
Dim ds As DataSet = New DataSet()
adapter.Fill(ds, "Customers")
DataGrid1.DataSource = ds
DataGrid1.DataBind()
```

2.3.5.3.2 UpdateCommand Metodu

```
Dim adapter As SqlDataAdapter = New SqlDataAdapter("SELECT
CategoryID, CategoryName FROM Categories", nwindConn)
adapter.UpdateCommand = New SqlCommand("UPDATE Categories SET
CategoryName = @CategoryName " & _
"WHERE CategoryID = @CategoryID", nwindConn)
adapter.UpdateCommand.Parameters.Add("@CategoryName",
SqlDbType.NVarChar, 15, "CategoryName")
Dim param As SqlParameter =
adapter.UpdateCommand.Parameters.Add("@CategoryID", SqlDbType.Int)
param.SourceColumn = "CategoryID"
param.SourceVersion = DataRowVersion.Original
Dim ds As DataSet = New DataSet
adapter.Fill(ds, "Categories")
Dim cRow As DataRow = ds.Tables("Categories").Rows(0)
cRow("CategoryName") = "AspNedir"
adapter.Update(ds)
```

2.3.5.3.3 InsertCommand Metodu

```
Dim adapter As SqlDataAdapter = New SqlDataAdapter("SELECT
CategoryID, CategoryName FROM Categories", nwindConn)
```

```
adapter.InsertCommand = New SqlCommand("Insert Into Categories
(CategoryName) Values @CategoryName", nwindConn)
adapter.UpdateCommand.Parameters.Add("@CategoryName",
SqlDbType.NVarChar, 15, "CategoryName")
Dim ds As DataSet = New DataSet
adapter.Fill(ds, "Categories")
Dim cRow As DataRow = ds.Tables("Categories").Rows(0)
cRow("CategoryName") = "BitirmeOdevi"
adapter.Update(ds)
```

2.3.5.3.4 DeleteCommand Metodu

```
Dim SqlIfadesi As String = "SELECT * FROM Categories ORDER BY
CategoryID"
Dim baglanti As String =
"server=localhost;uid=sa;pwd=;database=northwind"
Dim adapter As New SqlDataAdapter(SqlIfadesi, baglanti)
adapter.DeleteCommand.CommandText = "DELETE FROM Categories
WHERE CategoryName='BitirmeOdevi'"
adapter.DeleteCommand.Connection =
CType(adapter.SelectCommand.Connection, SqlConnection)
```

2.3.5.4 Data Reader

2.3.5.4.1 DataReader Nedir?

DataReader .NET platformunda karşımıza yeni çıkan ve herhangi bir kaynaktaki veriye FORWARD-ONLY ve READ-ONLY olarak ulaşan bir bileşendir. DataSet 'in belleği fazlaca kullanması bazı uygulamalarda sorun olabilir. Eğer veri tabanındaki kayıtlara salt okunur olarak erişmek yeterli oluyor veya veriler üzerinde bir değişiklik yapılması gerekmiyorsa Dataset yerine DataReader nesnesi ile çalışarak bellek daha verimli kullanılabilir. DataReader nesnesinin, sistem kaynakları kullanımı açısından, daha tutumlu olmasının diğer bir sebebi de; veri tabanından kayıtları sadece tek yönde, (forward only olarak) okumasıdır. Bu sayede herhangi bir t anında bellekte sadece bir kayıt bulunabilir. Bu çalışma tarzı bellek kullanımını önemli ölçüde azaltarak veri tabanı işlemlerine hız kazandırır.[7]

2.3.5.4.2 DataReader Oluşturulması

DataReader nesnesini oluşturmak için; Command nesnesinin **ExecuteReader()** isimli metodu kullanılabilir.

```
'Veritabanına bağlantı prosedürü hazırlanıyor.
Dim conn as New
SqlConnection("server=localhost;database=NorthWind;uid=sa;pwd=")
'Bir SQLCOMMAND hazırlanıyor.
```

```
Dim cmd as New SqlCommand()
'Ve Bir SQLDATAREADER hazırlanıyor.
Dim reader as SqlDataReader

'Bağlantı açılıyor.
Conn.open()
'Burada Command objesinin CommandText özelliğinde birlikte çalıştırmak
istediğimiz
'her iki query'i aralarına ; (noktalı virgül) koyarak yazılıyor.

cmd.CommandText ="Select count(*) from products;Select * from products"
cmd.Connection = conn

'SqlDataReader için query çalıştırılıyor.

reader = cmd.ExecuteReader()

Conn.Close()
```

2.3.5.4.3 Read Metodu

Veri tabanından bir kaydı (satırı) DataReader nesnesine okur. Read() metodu başarılı ise True, başarısız ise False değerini döndürür. Genellikle okumalar bu özellikten faydalanılarak bir (Do While Loop) döngüsü ile gerçekleştirilir. Döngü Read() metodundan dönen değer True olduğu sürece devam ettirilir. Kayıda ait field'lar ise, tek tek, bir dizi **GetXXX() metodu** ile elde edilir. [**GetString(), GetInt32(), GetDate(), GetDecimal()** gibi] GetXXX metotlarının parametresi field index'ini ifade eder.

Örnek: Aşağıda pubs veritabanındaki authors tablosundan, Data Reader 'a okunan veriler, ListView'da listelenmiştir.

```
Dim cnn As New SqlClient.SqlConnection()

cnn.Open()

Dim cmd As SqlClient.SqlCommand = New SqlClient.SqlCommand(sSQL,
cnn)

Dim oDr As SqlClient.SqlDataReader

oDr = cmd.ExecuteReader(CommandBehavior.CloseConnection)

Dim oLst As ListViewItem

Do While oDr.Read = True
```

```
oLst = New ListViewItem(oDr.GetString(1))  '2. field

oLst.SubItems.Add(oDr.GetString(2)) '3.field

oLst.SubItems.Add(oDr.GetString(4)) '5.field

ListView1.Items.Add(oLst)

oLst = Nothing

Loop

oDr.Close()

End Sub
```

2.3.5.5 Command Nesnesi

2.3.5.5.1 Command Nesnesi Nedir?

Command nesnesi, veri kaynaklarına karşı icra edilecek komutları temsil eden **ADO.NET** nesnesidir. **ADO**'daki Command nesnesi ile benzer özelliklere sahiptir. Command nesnesi de bir **Provider** nesnesidir ve temsil ettiği komutları icra ettirebilmesi için kurulmuş (açık) bir bağlantıya (Connection nesnesine) ihtiyacı vardır.

Command nesnesi neler yapabilir?
* Direk **SQL** sorguları yürütebilir. (*SELECT, INSERT, UPDATE, DELETE ve diğer DDL komutları CREATE, ALTER, ..*)
* **Depolanmış prosedür** (*Stored Procedure*) işletimi yaptırabilir.
* Depolanmış prosedürler için çeşitli değer aktarım tiplerinde (**INPUT, OUTPUT, RETURN VALUE**) parametreler bildirebilir.
* Yürütülen sorgu (veya stored procedure) sonucu dönen kayıtlara bir DataReader nesnesiyle ulaşılabilir.
* Direk bir tablo adını yazarak tüm kayıtlarını döndürebilir.[5]

2.3.5.5.2 Command Oluşturma ve Kullanma

Bir Command nesnesini, parametresiz sınıf yapılandırıcısı (*constructor*) ile oluşturmak mümkündür. System.Data.SqlClient sağlayıcısı ile SqlCommand nesnesi, System.Data.OleDb sağlayıcısı ile OleDbCommand nesnesi kullanılır.

```
Dim cmdTest as SqlCommand = New SqlCommand
```

```
Dim cmdTest as New OleDb.OleDbCommand = New
OleDb.OleDbCommand

CmdTest.CommandText = "Select * FROM Employees"
```

3. UYGULAMA

Windows uygulamalarının büyük bir kısmını formlar ve veritabanı işlemleri kaplar. Genelde kod kısmında da algoritmalar'dan ziyade kurulan veritabanı bağlantıları, veritabanından veri çekme, veritabanına veri kaydetme ve veri güncelleme gibi işlemlerin ağırlıklı olduğu kod parçaları bulunur. Aşağıda anlatacağım uygulamada klasik bir windows uygulamasıdır. Bu yüzden uygulamanın ana konusu kullanıcı arayüzünden girilen verileri veritabanına kaydetmek ve yukarıda saydığım diğer işlemleri yapmak olacak.

3.1 Kullanıcı Arayüzü

Uygulama çalıştırıldığı zaman kullanıcının karşısına ilk olarak yapabileceği işlemlerin listesini içeren bir menu formu çıkar. Kullanıcı bu formu kullanarak yapmak istediği işlemi menuden seçer. Kullanıcın seçimi doğrultusunda, karşısına çıkan form üzerinde yapmak istediği işlemleri yapar. Aşağıda bu formların nasıl tasarlandığı, nasıl kullanılacağı ve formun çalışmasını sağlayan önemli kod parçaları anlatılmıştır.

3.1.1 Menü Formu

Bu form uygulama çalıştırıldığı zaman kullanıcın karşısına çıkan ilk formdur ve diğer formlara bu form kullanılarak ulaşılır. Menu formu kapatıldığı zaman uygulama sonlanır.

3.1.1.1 Menü Formunun Görünümü ve Kullanımı

Şekil 3.1. Menu Formu

Yukarıdaki şekilde de görüldüğü gibi Menu formu Firma Rehberi, Eleman ve İş olmak üzere 3 menu de toplanmıştır.

Firma Rehberi: Bu menu altında Firma Kayıt, Firma Listesi ve Firma Takip gibi seçenekler bulunmaktadır. Firma kayıt seçeneği ile bir firma hakkındaki tüm kartvizit bilgilerini veritabanına kaydetmeye yarayan Firma Kayıt formuna ulaşılabilir. Firma Listesi ile de şu ana kadar kartvizit bilgileri girilmiş tüm firmaların listesini göstermeye yarayan Firma Listesi formuna geçiş yapılır. Firma Takip seçeneği kullanılarak da firmaların semt, sokak, sektör ve altsektör bazında filtrelenerek listelenmesini sağlayan Firma Takip formuna geçiş yapılabilir.

Eleman: Bu menude genel olarak şirkette çalışan elemanların bilgilerini görmeye ve kaydetmeye yarayan formlara geçiş yapan seçenekler bulunmaktadır. Eleman Kayıt seçeneği ile şirkete yeni işe girmiş bir elemanın bilgilerini kaydetmeye yarayan Eleman Kayıt formuna geçiş yapılır. Eleman Listele seçeneği ile şu ana kadar şirkette çalışmış tüm elemanların bilgilerini göstermeye yarayan Eleman Listele formuna geçiş yapılır. Eleman Maaş seçeneği ile Elemanlara verilen paraları tarih bazında kaydetmeye yarayan Eleman Maaş formuna geçiş yapılır. Eleman Takip seçeneği ile de elemanların performansını takip etmeye yarayan, yani hangi elemanın hangi tarihlerde kaç tane iş yaptığını ve hangi işleri yaptığını göstermeye yarayan Eleman Takip Formuna geçiş yapılır.

İş: Bu bölümde iş yerinde yapılan sözleşmeleri içeren ve bu sözleşmelerin takip edilmesine yarayan formlara geçiş sağlayan seçenekler bulunur. İş Kayıt seçeneği ile yapılan sözleşmelerin kaydedildiği Sözleşme formuna geçiş yapılır. İş Listesi seçeneği ile şu ana kadar yapılmış işlerin listesini gösteren İş Listesi formuna geçiş yapılır. İş Takip seçeneği ile de Tarih bazında yapılan sözleşmeleri filtrelemeye yaran İş Takip formuna geçiş yapılır.

3.1.1.2 Menu Formuna Ait Kod Açıklaması

Form üzerindeki menulerden herhangi birindeki seçeneklerden biri seçildiği zaman öncelikle o seçenek seçildiği zaman açılması istenen Form türünde yeni bir değişken tanımlanır ve bu değişkenin **Show** metodu kullanılarak formun aktif olması sağlanır.

Menülerdeki tüm seçenekler için benzer kod kullanıldığı için hepsinde benzer kodları tekrar etmektense sadece bir seçenek için kod parçasının anlatılması tercih edilmiştir.

```
Public Class frmMenu
   Inherits System.Windows.Forms.Form

Windows Form Designer Generated Code
Private Sub menuFKayit_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles menuFKayit.Click
      Dim frmFKayit As frmFirmaKayit = New frmFirmaKayit()
      frmFKayit.Show()
End Sub

End Class
```

Form dizayn edilirken kod bölümünde forma ait Class kendiliğinden oluşur ve form üzerine eklediğimiz tüm kontroller de kod bölümüne otomatik olarak eklenir. *Windows Form Designer Generated Code* kısmında bu kontrolleri temsil eden kodlar otomatik olarak bulunmaktadır.

Yukarıdaki kod parçasında menuFKayit seçeneğinin **Click** olayında(event) frmFKayit adında frmFirmaKayit formu tipinde bir değişken tanımlanmıştır. Sonrada bu değişkenin *Show* özelliği kullanılarak formun gösterilmesi sağlanmıştır. Böylece kullanıcı menuFKayit adındaki(Firma Kayit) seçeneğine Click yapınca kod çalışıyor ve Yeni bir pencere açılıyor.

3.1.2 Firma Kayıt Formu

Bu form kullanılarak firmaların kartvizit bilgileri veritabanına ekleniyor. Ayrıca daha önceden kaydedilmiş firmaların verilerine ulaşmak için de bu formdan bir geçiş bulunmaktadır.

3.1.2.1 Firma Kayıt Formunun Görünümü ve Kullanımı

Firma Kayıt formu üzerinde şirket adı, hangi sektörde çalıştığı, hangi semtte bulunduğu gibi bilgileri seçmemize yarayan ComboBox'lar ve, şirketin yetkilisnin adını, telefon ve fax numarası gibi bilgileri girmemize yarayan TextBox gibi kontroller bulunmaktadır.

Şekil 3.2. Şirket Kayıt Formu

Ekle: Ekle menüsünde *Sektör Ekleme, Alt Sektör Ekleme, Semt Ekleme ve Sokak Ekleme* gibi seçenekler bulunmaktadır. *Sektör Ekleme* seçeneği kullanılarak herhangi bir şirketin hangi sektörde çalıştığını gösteren, veritabanındaki sektör toblosuna yeni bir veri eklemeye yarayan **Sektör Kayıt Formu** açılır. *Alt Sektör Ekleme* seçeneği ilede herhangi bir sektörün altına alt sektör eklemek için kullanılan **Alt Sektör Kayıt** formuna geçiş yapılır. *Semt Ekleme ve Sokak Ekleme* seçenekleriyle de aynı şekilde semt ve seçilen semte sokak eklemek için kullanılan **Semt Kayıt** ve **Sokak Kayıt** formlarına geçiş yapılabilir.

: Şirket adı yanındaki bu buton kullanılarak Tüm şirketlerin listesi bulunan yeni bir forma geçiş yapılabilir ve yeni formdaki herhangi bir şirketin üzerine tıklayınca tekrar **Şirket Kayıt Formu'** na dönüldüğünde üzerine tıklanan şirketin detaylı bilgilerine ulaşılabilir. Form ilk açıldığında aktif olmayan *ileri* ve *geri* butonları kullanılarak veritabanında seçilen şirketin altında ve üstünde bulunan diğer şirketlerin de detaylı bilgilerine ulaşılır.

Formun alt tarafındaki butonlar kullanılarak yeni bir kayıt yapılabilir, varolan kayıtlar üzerinde ileri geri hareket edilebilir ve istenilen bir kayıt güncellenebilir ya da silinebilir.

Butonlar: Formun alt tarfında solda sağa sırasıyla *yeni kayıt, kaydet, ilk kayıt, geri, ileri, son kayıt, güncelle* ve *sil* butonları bulunmaktadır.

: Form üzerinde yeni bir kayıt yapabilmek için ilk önce bu butonun tıklanması gerekmektedir. Bu buton tıklandığı zaman form temizlenir, yani form üzerindeki tüm TextBox'ların içeriği silinir ve yeni bir kayıt için hazır hale getirilir. Bu buton tıklandığı zaman *kaydet* butonu aktif hale gelir ve kullanıcı verileri girdikten sonra *kaydet* butonuna tıklayarak form üzerine girdiği verileri veritabanına kaydeder. Ayrıca bu buton ilk tıklandığında *kaydet* haricindeki butonlar pasif duruma geçer. Diğer butonların aktif hale geçebilmesi için Şirket Adı'nın yazıldığı TextBox'ın yanındaki ara butonuna tıklanması gerekmektedir. Çünkü diğer butonlar veritabanında zaten kayıtlı veriler üzerinde işlem yapmak için kullanılan butonlardır.

: Kayıt butonu form ilk açıldığında pasiftir ve yeni bir kayıt yapılmak için yeni kayıt butonuna tıklandığı zaman aktif hale gelir. Form üzerindeki bilgiler doldurulduktan sonra bu butona tıklandığı zaman, ilk önce doldurulmayan bir TextBox veya seçilmeyen bir ComboBox olup olmadığı kontrol edilir ve eğer varsa kullanıcıya bu eksiklikler bir mesaj kutusu ile haber verilir ve kullanıcıya kayıt işlemine devam edip etmeyeceği sorulur. Kayıt işlemi tamamlandıktan sonra form üzerindeki TextBox'lar temizlenir.

Şekil 3.3. Uyarı Mesajları

: İlk kayıt butonu, kullanıcı herhangi bir kaydın detayını görmek için Ara butonuna tıkladığı zaman aktif olan butanlardan biridir ve veri tabanındaki ilk kayda ulaşmak için kullanılır.

: Geri butonu, veritabanındaki kayıtlardan, şu an form üzerinde detayları bulunan kaydın üstündeki kayıtların detaylarını getirmek için kullanılır.

: İleri butonu, veritabanındaki kayıtlardan, şu an form üzerinde detayları bulunan kaydın altındaki kayıtların detaylarını getirmek için kullanılır. Bu butonun aktif olabilmesi için kullanıcının öncelikle bir kaydın detaylarını görmek için Ara butonuna tıklaması gerekmektedir.

: Son butonu veritabanına eklenen son kayıtla ilgili detayları form üzerine getirmek için kullanılır.

: Veritabanına detayları getirilen firmanın bilgileri üzerinde herhangi bir değişiklik yapılmak istenirse, değiştirilmek bilgi değiştirilir ve Güncelle butonuna tıklanır.

: Form üzerinde detayları görünen kaydı silmek için kullanılır.

3.1.2.2 Firma Kayıt Formuna Ait Kod Açıklaması

Uygulama genelinde kullanılan kısaltmalar:
cbo: ComboBox
txt : TextBox
menu : MenuItem
lbl : Label
tlb : ToolBar
frm : Form

3.1.2.2.1 Form_Load

Bir form ilk ekrana geldiği zaman çalışmasını istediğimiz kod parçasını bu form_load prosedürü içerisine yazarız. Bu prosedür bir Sub prosedürdür ve geriye herhangi birşey döndürmez.

Şirket kayıt formu ilk açıldığında Semt ve Sektor seçmek için kullanılan ComboBox'ların içeriğinin dolu oması gerekmektedir. Ayrıca Semt seçildikten sonra Sokak ve Sektör seçildikten sonra da AltSektör'ü temsil eden ComboBox'ların içeriği dolmaktadır.

<u>Kod Parçası:</u>

```
Private Sub Form1_Load(ByVal sender As System.Object,...)
Dim objDV2 As DataView = New DataView()
  objDA.Fill(objDS, "tblsemt")
  objDA.SelectCommand.CommandText = "Select * From tblSektor"
  objDA.Fill(objDS, "tblsektor")
  objDV = objDS.Tables("tblsemt").DefaultView
  objDV2 = objDS.Tables("tblsektor").DefaultView
  cboSemt.DataSource = objDV
  cboSemt.DisplayMember = "semtadi"
  cboSemt.ValueMember = "semtid"
  cboSektor.DataSource = objDV2
  cboSektor.DisplayMember = "sektoradi"
  cboSektor.ValueMember = "sektorid"
  btnKaydet.Enabled = False
  btnGeri.Enabled = False
  btnIleri.Enabled = False
  btnIlk.Enabled = False
  btnSon.Enabled = False
End Sub
```

Kod Açıklaması:
Öncelikle objDA adında bir Data Adapter, objDV , objDV2 adında iki tane Data View ve objDS adında da bir Data Set tanımlanmıştır.
Dim *objDV2* ***As*** *DataView =* ***New*** *DataView()*

Daha sonra objDA nesnesinin Fill Metodu kullanılarak objDS dataset'inin içine semt bilgilerini içeren yeni bir tablo ekleniyor.
objDA.Fill(objDS, "tblsemt")

objDA nesnesinin SelectCommand metodunun CommandText özelliğine yeni bir SQL string atanıyor ve bu string sayesinde veritabanından sektörlerin bulunduğu tablodaki tüm veriler çekiliyor. Daha sonra tekrar ObjDA nesnesinin Fill Metodu kullanılarak objDS dataset'ine yeni bir tablo daha ekleniyor.
*objDA.SelectCommand.CommandText = "Select * From tblSektor"*
objDA.Fill(objDS, "tblsektor")

ComboBox: Bu nesne kullanılarak kullanıcılara tercih yapabilceği bir yapı sağlanır. ComboBox nesnesinin DataSource özelliği kullanılarak veritabanında tutulan bilgiler sergilenebilir. DisplayMember özelliği, DataSource olarak belirtilen tablonun hangi sütununun gösterileceğini belirlenir. ValuMember özelliği ise ComboBox'tan seçilen bir verinin değerinin ne olduğunu belirler.

OdjDV ve objDV2 nesnesi kullanılarak objDS dataset'in içerisindeki tabloların bir görüntüsü alınıyor ve Sektor ve Semt bilgilerini sergileyen ComboBox'ların DataSource özelliğine atanıyor. cboSektor combobox'ın DisplayMember özelliğine "sektoradi" ve ValueMember özelliğine de "sektorid" atanıyor. Aynı şekilde cboSemt nesnesinin DisplayMember özelliğine "semtadi" ve ValueMember özelliğine de "semtid" atanıyor. Böylece veritabanında sektörleri tutan tablodaki "sektoradi" sütunu cboSektor nesnesinde, semt bilgilerini tutan tablodaki "semtadi" sütunu da cboSemt nesnesinde sergilenmiş oluyor.
cboSemt.DataSource = objDV
cboSemt.DisplayMember = "semtadi"
cboSemt.ValueMember = "semtid"
cboSektor.DataSource = objDV2
cboSektor.DisplayMember = "sektoradi"
cboSektor.ValueMember = "sektorid"

Form ilk yüklendiği zaman btnKaydet, btnGeri, btnIleri, btnIlk, btnSon, btnGuncelle, btnSil butonları pasif hale getiriliyor.

btnKaydet.Enabled = ***False***
btnIleri.Enabled = ***False***
btnIlk.Enabled = ***False***
btnGeri.Enabled = ***False***

*btnSon.Enabled = **False***
*btnGuncelle.Enabled = **False***
*btnSil.Enabled = **False***

3.1.2.2.2 cboSemt_Click

Semt isimlerini sergileyen cboSemt nesnesinin Click olayına kod yazılmıştır. Çünkü kullanıcı herhangi bir semt seçtiği zaman sokak bilgisinin bu semte bağlı olarak gelmesi istenmektedir.

Kod Parçası:

```
Private Sub cboSemt_Click(ByVal sender As Object, ...)
Dim DV1 As DataView = New DataView()
Dim Str As String
Str = "SELECT * FROM tblsokak WHERE " & _
"semtid= " & cboSemt.SelectedValue & ""
Dim DA1 As OleDb.OleDbDataAdapter = New
 OleDb.OleDbDataAdapter(Str, objConn)
Dim DS1 As DataSet = New DataSet()
DA1.Fill(DS1, "tblsokak")
DV1 = DS1.Tables("tblsokak").DefaultView
cboSokak.DataSource = DV1
cboSokak.DisplayMember = "sokakadi"
cboSokak.ValueMember = "sokakid"
objConn.Close()
End Sub
```

3.1.2.2.3 btnKaydet_Click

Kaydet butonunun Click olayına yazılan kod parçası ile öncelikle form üzerindeki verilerin eksiksiz doldurulup doldurulmadığı kontrol ediliyor ve doldrulmayan kısımlar kullanıcıya uyari mesajı olarak gösteriliyor, eğer kullanıcı bu eksikliklere rağmen kayıt işlemine devam etmek istiyorsa kayıt işlemi tamamlanıyor ve form üzerindeki veriler veritabanına kaydediliyor. Form üzerinde boş bırakılan kısımlara “???” işareti eklenerek kayıt yapılıyor.

Kod Parçası:

```
    Dim str As String
Dim msg As String = vbNewLine
Dim uyari As Integer
Dim cmd As OleDb.OleDbCommand = New OleDb.OleDbCommand()
If txtSirketAdi.Text = "" Then
 MessageBox.Show("Lütfen Sirket Adini Giriniz", "Uyari",
      MessageBoxButtons.OK)
```

```
        GoTo finish
        End If
        If txtYetkili.Text = "" Then
          txtYetkili.Text = "???"
          uyari = 1
          msg &= "Yetkili"
       End If
       If txtTelefon.Text = "" Then
         txtTelefon.Text = 111
         uyari = 1
         msg &= vbNewLine & "Telefon Numarasi"
      End If
      If txtFax.Text = "" Then
        txtFax.Text = "111"
        uyari = 1
        msg &= vbNewLine & "Fax Numarasi"
      End If
       If txtAdres.Text = "" Then
         txtAdres.Text = "???"
        uyari = 1
        msg &= vbNewLine & "Adres"
      End If
       If txtEmail.Text = "" Then
          txtEmail.Text = "???"
          uyari = 1
         msg &= vbNewLine & "Email"
      End If
      If txtWeb.Text = "" Then
        txtWeb.Text = "???"
        uyari = 1
        msg &= vbNewLine & "Web" & vbNewLine
      End If
      If uyari = 1 Then
            MessageBox.Show("Asagidaki Alanlari Doldurmadiniz" & msg & "Devam
            Etmek Istiyormusunuz?", "Uyari Mesaji", MessageBoxButtons.YesNoCancel,
            MessageBoxIcon.Warning)
       If vbNo = MsgBoxResult.No Or vbCancel = MsgBoxResult.Cancel Then
              GoTo finish
            End If
     End If

str = "INSERT INTO
            tblmusteri(sirketadi,sektor,altsektor,yetkili,telefon,fax,semt,sokak,adres,web,email
            )" & _
        "VALUES(" & _
                " '" & txtSirketAdi.Text & "'" & _
                ", " & cboSektor.SelectedValue & _
```

```
            ", " & cboAltSektor.SelectedValue & _
            ", '" & txtYetkili.Text & "'" & _
            ", '" & txtTelefon.Text & "'" & _
            ", '" & txtFax.Text & "' " & _
            ", " & cboSemt.SelectedValue & _
            ", " & cboSokak.SelectedValue & _
            ", '" & txtAdres.Text & "'" & _
            ", '" & txtWeb.Text & "'" & _
            ", '" & txtEmail.Text & "'" & ")"
    cmd.CommandText = str
    cmd.Connection = objConn
    objConn.Open()
    cmd.ExecuteNonQuery()
    objConn.Close()
MsgBox("Kayit tamamlanmistir")
        finish:

        Temizle()

        End Sub
```

3.1.3 Firma Listele Formu

Bu form kullanılarak firmaların tutulduğu tabloda Semt, Sokak, Sektör ve Altsektör bazında filtreleme yapılarak veriler listelenebiliyor.

3.1.3.1 Firma Listele Formunun Görünümü ve Kullanımı

Şekil 3.4. Firma Listele Formu

Yukarıdaki şekilde görüldüğü filtrelenmiş verileri göstermek için DataGrid nesnesi kullanılmıştır. Kullanıcı öncelikle listelemek istediği firmalar için Semt seçecek. Eğer sadece semt seçilmişse, mesela semt olarak “Osmanbey” seçilmiş ve diğer seçenekler kullanılmamışsa Osmanbeydeki tüm firmalar listelenir. Eğer Semt ve Sektör seçilmişse seçilen semtte, seçilen sektörde çalışan firmalar listelenir.

3.1.3.2 Şirket Kayıt Formuna Ait Kod Parçası

Firma Kayıt formu anlatılırken kod parçaları ile detaylı açıklama yapılmıştı. Bu bölümde Firma Kayıt formunda açıklanan kod parçalarına benzer kodlar için açıklama ihtiyacı duyulmamıştır, bu yüzden bazı kodlar açıklamasız bir şekilde aktarılmıştır.

3.1.3.2.1 frmFirmaListele_Load

Kod Parçası:

```
Private Sub frmFirmaListele_Load(ByVal sender As System.Object, ...)

Dim strListele As String
strListele = "Select * From tblsemt"
daListele = New OleDb.OleDbDataAdapter(strListele, conn)
dsListele = New DataSet()
daListele.Fill(dsListele, "tblsemt")
daListele.SelectCommand.CommandText = "Select * From tblsektor"
daListele.Fill(dsListele, "tblsektor")
dvSemt = dsListele.Tables("tblsemt").DefaultView
dvSektor = dsListele.Tables("tblsektor").DefaultView
cboSektor.DataSource = dvSektor
cboSektor.DisplayMember = "sektoradi"
cboSektor.ValueMember = "sektorid"
cboSemt.DataSource = dvSemt
cboSemt.DisplayMember = "semtaadi”
cboSemt.ValueMember = "semtid"

End Sub
```

3.1.3.2.2 btnListele_Click

Kod Parçası

```
Private Sub btnListele_Click(ByVal sender As System.Object, ...)

strListele = "Select * From tblmusteri Where"
    If cboSemt.Text <> "" Then
      strListele &= " semt = " & cboSemt.SelectedValue & ""
    End If
    Dim ve As String = "And"
        If cboSokak.Text <> "" Then
            strListele &= ve & " sokak = " & cboSokak.SelectedValue & ""
    End If
    If cboSektor.Text <> "" Then
      strListele &= ve & " sektor = " & cboSektor.SelectedValue & ""
    End If
    If cboAltSektor.Text <> "" Then
      strListele &= ve & " altsektor = " & cboAltSektor.SelectedValue & ""
    End If
    daListele.SelectCommand.CommandText = strListele
    daListele.FillSchema(dsListele, SchemaType.Source, "tblliste")
    daListele.Fill(dsListele, "tbllistele")
```

```
    DataGrid1.SetDataBinding(dsListele, "tbllistele")

End Sub
```

Kod Açıklaması:

Bu kodun amacı kullanıcı Kayıt Listele butonuna tıkladığı zaman kullanıcın belirlediği kriterlere göre, veritabanında kayıtlı bulunan firmaların listesini getirmektir. Bu yüzden öncelikle kullanıcın hangi seçenekleri yaptığı kontrol ediliyor ve ComboBox'lardan seçilen

Daha sonra veritabanından bilgileri çekmemize yarayan SQL stringi oluşturuluyor ve bu string daListele adındaki DataAdapter nesnesinin SelectCommand metodunun CommandText özelliğine atanıyor. daListele nesnesinin Fill metodu kullanılarak dsListele DataSet nesnesine istenilen veriler aktarılıyor. Ve DataGrid nesnesinin SetDataBinding metodu kullanılarak veriler DataGrid üzerinde sergileniyor.

```
daListele.SelectCommand.CommandText = strListele
daListele.FillSchema(dsListele, SchemaType.Source, "tblliste")
daListele.Fill(dsListele, "tbllistele")

DataGrid1.SetDataBinding(dsListele, "tbllistele")
```

3.1.4 Eleman Kayıt Formu

Bu form kullanılarak Şirkette çalışan elemanlara ait bilgiler veritabanına kaydedilir.

Şekil 3.4. Eleman Kayıt Formu

Eleman Kayıt Formu hazırlanırken TabControl kullanılmıştır, bu sayede form kişisel bildiler ve iş bilgileri olmak üzere iki bölümde gösterilmiştir. Kullanıcı Kişisel bilgilere tıklayarak zaman Kişisel Bilgiler formunu, İş Bilgileri kısmına tıklayarak da Mesleki bilgilerle ilgili formu doldurabilir. İki kısım da doldurulduktan sonra kayıt yapabilir.

Bu formda da daha önceki formlarda olduğu gibi yeni kayıt butonuna tıklanarak form yeni bir kayıt için hazır hale getirilir ve form üzerindeki kutular doldurulduktan sonra kayıt butonu kullanılarak bilgiler kaydedilir. Aynı şekilde ilk kayıt, son kayıt, ileri ve geri gibi butonlar kullanılarak veritabanındaki daha önceden kaydedilmiş elemanlara ait bilgilerin detayları gösterilebilir.

AdSoyad bilgisinin girildiği TextBox'ın yanındaki buton kullanılarak daha önceden kaydedilmiş elemanların listesi görülebilir ve listeden seçilen elemana ait bilgiler Eleman Kayıt formunda sergilenir.

Form ilk yüklendiği zaman Yeni kayıt butonu aktif, Kaydet, İlk Kayıt, Son Kayıt, İleri ve Geri butonları ise pasif durumdadır. Kaydet butonunun aktif olabilmesi için öncelikle yenikayıt butonuna tıklanması gerekmektedir. Ileri,Geri, Ilk Kayıt, Son Kayıt butonlarının aktif olabilmesi için ise Adsoyad kısmının yanındaki Ara butonunun tıklanması gerekmektedir.

3.1.5 Sözleşme Kayıt Formu

Bu form kullanılarak satış elemanlarının yapmış olduğu sözleşmeler veritabanına kaydedilir.

Şekil 3.5. Sözleşme Kayıt Formu

Sözleşme kayıt formu kullanılarak yeni yapılmış sözleşmeler veritabanına kaydedilir. Sözleşme yapılan firma adı girilir ve sözleşmeyi kimin yaptığı ComboBox'tan seçilir. Daha sonra Sözleşmenin başlangıç ve bitiş tarihi bir Calender yardımıyla seçilir. Ayrıca form üzerinde sözleşmenin toplam nekadar ücrete yapıldığu, alınan peşinat ve kalan paranın alım tarihi gibi bilgiler bulunmaktadır. Kullanıcının isterse firma hakkında yorum yazabilmesi için Firma Hakkında kısmı kullanılabilir.

Bu formda da form ilk yüklendiği zaman Yeni kayıt ve Ara butonu dışındaki butonlar pasif durumdadır. Veritabanına daha önceden kaydedilmiş verilerin detaylı bilgilerini görebilmek için kullanılan *ileri*, *geri*, *ilk* ve *son* butonları *Firma Adı* kutucuğunun yanındaki Ara butonuna tıklandığı zaman aktif hale gelir.

Ara butonu kullanılarak herhangi bir firmanın detaylı bilgileri Form üzerine getirildikten sonra ileri ve geri tuşları kullanılarak veritabanındaki diğer firma bilgilerinin detayları form üzerinde sergilenebilir.

3.2 Veritabanı Tabloları

Veritabanında bilgileri tutmak için toplam 7 tane tablo oluşturulmuştur: *tblsektor*, *tblaltsektor, tblsemt, tblsokak, tbleleman, tblsozlesme*.

3.2.1 Sektör Tablosu (tblsektor)

Bu tabloda Tekstil işine ait sektor bilgileri bulunmaktadır.

Şekil 3.6. Sektör Tablosu

Sektör tablosu (tblsektor) ile kullanıcılar frmSektorKayıt formunu kullanarak sektör bilgilerini depoluyorlar. Bu bilgiler proğram ilk çalıştırıldığı zaman girilirse, daha sonraki proğram işlemlerinde kullanılabilir. Sektör taplosu kullanılarak sektör isimleri bir ComboBox içerisinde sergilenir ve böylece kullanıcı her yeni şirket kayıt işleminde tekrardan sektör ismi girmek zorunda kalmaz.

Sektorid: AutoNumber tipinde bir değişkendir ve birincil anahtar(primary key) olarak kullanılır.
Sektoradi: Text tipinde bir değişkendir ve Sektör adlarını tutar.

3.2.2 Alt Sektör Tablosu(tblaltsektor)

Şekil 3.7. Alt Sektör Tablosu

Alt Sektör Tablosu(tblaltsektor) daha önceden kaydedilmiş sektor isimlerine ait altsektorleri tutmak için kullanılır. Her altsektorün bir sektörü vardır ve her sektör birden fazla alt sektöre sahip olabilir. FrmAltSektor formu kullanılarak, öncelikle sektör isimlerini sergileyen ComboBox içerisinden bir sektör seçilir ve daha sonra bu sektöre ait yeni bir alt sektör kaydedilir. Böylece daha sonra şirketlere ait kartvizit bilgileri girileceği zaman herseferinde altsektör adı yazılmak zorunda kalınmaz ve alt sektör isimlerinin sergilendiği ComboBox içerisinden seçilir.

altsektorid: AutoNumber tipinde bir değişkendir ve birincil anahtar(primary key) olarak atanmıştır.
sektorid: Altsektörün hangi sektöre ait olduğunu tutan Number tipinde bir değişkendir.
altsektoradi: Kullanıcının girdiği alt sektör adını tutan Text tipinde bir değişkendir.

3.2.3 Semt Tablosu(tblsemt)

Şekil 3.8. Semt Tablosu

Semt Tablosu(tblsemt) firmalara ait semt bilgilerini tutmak için kullanılır ve frmSemtEkle formu kullanılarak bu tabloya yeni bir kayıt eklenebilir. Böylece tüm semt bilgileri proğram ilk çalıştırıldığı zaman bir seferberliğine tabloya kaydedilirse, sonraki firma kayıt işlemlerinde tekrar tekrar firmanın bulunduğu semt ismini yazmak zorunda kalmayız, ve bu semt isimlerini içeren bir ComboBox içinden seçeriz.

Semtid: AutoNumber tipinde bir değişkendir ve birincil anahtar(primary key) olarak seçilmiştir.
Semtadi: Firmaların bulunduğu semt ismilerini tutan Text tipinde bir değişkendir.

3.2.4 Sokak Tablosu(tblsokak)

Şekil 3.9 Sokak Tablosu

Firmaların bulunduğu sokak isimleri bu tabloda tutulur ve frmSokakEkle formu kullanılarak, öncelikle semt isimlerinin listelendiği ComboBox kullanılarak hangi semte ait olduğu seçilir ve sonrada sokak ismi yazılarak kayıt yapılır.

Sokakid: AutoNumber tipinde bir değişkendir ve birincil anahtar olarak atanmıştır.
Semtid: Sokağın hangi semtte bulunduğu gösteren Number tipinde bir değişkendir.
Sokakadi: Kaydedilen sokağın ismini tutan Text tipinde bir değişkendir.

3.2.5 Eleman Tablosu(tbleleman)

Şekil 3.10. Eleman Tablosu

Bu tablo şirkette çalışan elemanlara ait bilgileri tutmak için kullanılır.
Elemanid: AutoNumber tipinde bir değişkendir ve birincil anahtar olarak(primary key) atanmışltır.
Adsoyad: Elemanın adını ve soyadını tutan Text tipinde bir değişkendir.
Adres: Elemanın oturduğu yerin adres bilgisini tutan Text tipinde bir değişkendir.
Evtel: Elemanın ev telefon numarasını tutan Number tipinde bir değişkendir.
Gsm: Elemnın cep telefon numarasını tutan Number tipinde bir değişkendir.
Departman: Elemanın şirkette hangi departmanda çalıştığı bilgisini tutan Number tipinde bir değişkendir.
Bastarih: Elemanın işe başlama tarihini tutan Date/Time tipinde bir değişkendir.
Maas: Elemanın ne kadar maaş aldığı bilgisini tutan bir değişkendir.
Email: Elemanın email adresini tutan Text tipinde bir değişkendir.

3.2.6 Sözleşme Tablosu(tblsozlesme)

Şekil 3.11 Sözleşme Tablosu

Yapılan sözleşmelere ait bilgilerin tutulduğu tablodur. Bu tabloya frmŞirketKayit formu kullanılarak veri kaydedilebilir.

sozlesmeid: AutoNumber tipinde bir değişkendir ve birincil anahtar olarak atanmıştır.
firmaid: Bu sözleşmenin hangi firmaya ait olduğu bilgisini tutan Number tipinde bir değişkendir ve aynı zamanda firma bilgilerini temsil eden bir dış anahtar(foreign key)'dır.
elemanid: Sözleşmeyi yapan elemana ait bilgileri temsil eden bir dış anahtardır ve number tipinde bir değişkendir.
baslangic: Sözleşmenin başlangış tarihini temsil eden Date/Time tipinde bir değişkendir.
bitis: Sözleşmenin bitiş tarihini temsil eden Date/Time tipinde bir değişkendir.
ucret: Sözleşmenin toplam ne kadar ücret karşılığı yapıldığını temsil eden bir değişkendir.
pesinat: Müşteriden alınan peşin para miktarını temsil eden bir değişkendir.
odemetarihi: Kalan paranın müşteriden tahsil edileceği tarihi temsil eden bir değişkendir.
hakkinda: Firma hakkında yapılan yorumları tutmak için kullanılan Memo tipinde bir değişkendir.

3.2.7 Müşteri Tablosu(tblmusteri)

Şekil 3.12. Müşteri Tablosu-I

Şekil 3.13. Müşteri Tablosu-II

Firmalar ait kartvizit bilgilerinin tutulduğu tablodur. Bu tabloya veriler frmSirketKayit formu kullanılarak kaydedilir.

sirketid: AutoNumber tipinde bir değişkendir ve birincil anahtar olarak atanmıştır.
sirketadi: Şirketin ismini tutan Text tipinde bir değişkendir.
sektor: Şirketin hangi sektörde olduğunu temsil eden Number tipinde bir dış anahtardır.
altsektor: Şirketin altsektörünü temsil eden bir dış anahtardır.
yetkili: Şirketteki Yetkili kişinin ismini tutan Text tipinde bir değişkendir.
telefon: Şirketlerin telefon numarasını tutan Number tipinde bir değişkendir.
fax: Şirketlerin fax numarasını tutan Number tipinde bir değişkendir.
semt: Şirketin hangi semtte olduğunu temsil eden Number tipinde bir dış anahtardır.
sokak: Şirketlerin hangi sokakta olduğunu temsil eden Number tipinde bir dış anahtardır.

adres: Şirketin semt ve sokak dışındaki adres kısımlarını tutan Text tipinde bir değişkendir.
web: Şirketin web adresini tutan text tipinde bir değişkendir.
email: Şirketin emailini tutan text tipinde bir değişkendir.

3.3 Tablolar Arası İlişkiler

Şekil 3.13. RelationShips

4 – SONUÇ VE ÖNERİLER

Projenin yapım aşamasında daha fazla deneyim ve bilgiye sahip olabilmek için mümkün olduğunca değişik nesneler ve kontroller kullanılmıştır. Ayrıca kullanılan Nesnelerin bazı özellikleri proje içerisinde kullanılmış olsa da hemen hemen tüm özellikleri incelenmiştir.

Projenin yapım aşamasında kaynak sıkıntısı çekildiğinden dolayı, proje boyunca Visual Studio 6.0 a dayanan deneyimlerin yardımıyla deneme yanılma yöntemi ve Visual Studio .Net'in help ksımındaki dökümantasyonlar kullanılmıştır. Özellikle de Visual Studio .Net'in Dynamic Help özelliği projenin yapım aşamasında ençok başvurulan kaynak olmuştur. Tabii elimizde yeteri kadar Türkçe kaynak bulunmaması projenin tamamlanma süresini uzatmıştır.

Projenin yapımı sırasında SQL Server'da oluşturulmuş veritabanlarının taşınması konusunda çekilen sıkıntıdan dolayı Access veritabanı kullanılmıştır. Tabii veritabanı access olunca Visual Basic .Net'ten veriye ulaşabilmek için OleDb nesneleri kullanılmak zorunda kalınmıştır. Her nekadar çok geniş kapsamlı bir proje olmasa da, bu durum projeyi performans açısından kötü yönde etkilemiştir. Projenin gelişim aşamasında SQL Server kullanılacaktır. Visual Studio .Net kullanarak veritabanı bağlantılı bir proje geliştirilmek istenirse SQL Server kullanılması önerilir.

Rapor hazırlama işine projenin bitimine doğru başlanmıştır. Rapor hazırlamak için döküman araştırmasına başlandığı zaman, aslında projede üzerinde çok durulan ve zaman kaybına sebep olan bazı kısımların çok basit ve daha etkili yöntemlerle yapılabileceği farkedilmiştir. Proje geliştirecek arkadaşlara projenin yapımına başlanmasıyla birlikte rapor hazırlıklarına da başlanması önerilir.

Raporun giriş bölümünde de belirtildiği gibi bu proje seçilirken ana amaç Windows uygulamaları hakkında deneyim sahibi olabilmek , Visual Studio .Net platformu ve Visual Basic .Net hakkında bilgi sahibi olabilmekti ve projenin sonuna gelinen şu aşamada projeye başlamadan önceki amaca ulaşıldığı söylenebilir.

5 - KAYNAKLAR

[1] http://www.aspnedir.com/sqlservergenel.htm

[2] http://www.aspnedir.com/adonetozellik.htm

[3] http://www.aspnedir.com/adonetavantaj.htm

[4] http://www.msakedemik.net/adoyagiris.htm

[5] http://www.evcilnet.com/Evcil_NET _ Connection (Bağlantı)Nesnesi.htm

[6] http://www.msakedemik.net/datasetnedir.htm

[7] http://www.msakedemik.net/datareadernedir.htm